

Fundamentals of theory of computation I.

Lecture 5

Krisztián Tichler
ktichler@inf.elte.hu

NFA to DFA

Theorem

For every nondeterministic finite automaton $A = (Q, \Sigma, \delta, Q_0, F)$ there exists a deterministic finite automaton $A' = (Q', \Sigma, \delta', q'_0, F')$ with the property $L(A) = L(A')$.

Proof:

Construction:

$$Q' := \mathcal{P}(Q), \quad q'_0 := Q_0, \quad F' := \{q' \in Q' \mid q' \cap F \neq \emptyset\},$$

$$\delta'(q', a) := \bigcup_{q \in q'} \delta(q, a).$$

NFA to DFA

Lemma 1

For every $p, q \in Q$, $q' \in Q'$ and $u, v \in \Sigma^*$

$$qu \Rightarrow_A^* pv \text{ and } q \in q'$$

imply

$$q'u \Rightarrow_A^* p'v \text{ and } p \in p'$$

for some $p' \in Q$.

Proof of Lemma 1:

The proof is by induction on the number of steps in the reduction $qu \Rightarrow^* pv$.

If $qu \Rightarrow^* pv$ is a 0 step reduction then we have $q = p$. So the 0 step reduction and $p' = q'$ makes the statement hold.

Let us assume, that the statement holds for reductions of at most n steps ($n \geq 0$).

NFA to DFA

Assume, that a reduction $qu \Rightarrow_A^* pv$ consists of $n + 1$ steps. Then we have

$$qu \Rightarrow_A q_1 u_1 \Rightarrow_A^* pv$$

for some $q_1 \in Q$ and $u_1 \in \Sigma^*$. So, there is a $a \in \Sigma$ with $u = au_1$ and $q_1 \in \delta(q, a)$.

By $q \in q'$ and the definition of δ' we have $\delta(q, a) \subseteq \delta'(q', a)$.

Let $q'_1 = \delta'(q', a)$ implying

$$q' u \Rightarrow_{A'} q'_1 u_1 \tag{1}$$

and $q_1 \in \delta(q, a) \subseteq \delta'(q', a) = q'_1$, so we have $q_1 \in q'_1$.

By induction we have

$$q'_1 u_1 \Rightarrow_{A'}^* p' v \text{ and } p \in p'. \tag{2}$$

for some $p' \in Q'$.

The statement of Lemma 1 holds by (1) and (2).

NFA to DFA

Lemma 1 implies $L(A) \subseteq L(A')$:

Let $u \in L(A)$, i.e., $q_0 u \Rightarrow_A^* p$ holds for some $q_0 \in Q_0$ and $p \in F$.

By Lemma 1 $q'_0 u \Rightarrow_{A'}^* p'$ and $p \in p'$ holds for some $p' \in Q'$.

By the definition of F' , $p' \in F'$ and $p \in p'$ we have $p' \in F'$ implying $u \in L(A')$.

As $u \in L(A)$ has been chosen arbitrarily we have $L(A) \subseteq L(A')$.

NFA to DFA

Lemma 2

For every $p', q' \in Q'$, $p \in Q$ and $u, v \in \Sigma^*$

$$q'u \Rightarrow_{A'}^* p'v \text{ and } p \in p',$$

imply

$$qu \Rightarrow_A^* pv \text{ and } q \in q'$$

for some $q \in Q$.

Proof of Lemma 2: .

The proof is by induction on the number of steps in the reduction $q'u \Rightarrow_{A'}^* p'v$.

If $q'u \Rightarrow^* p'v$ is a 0 step reduction then we have $q' = p'$. So the 0 step reduction and $q = p$ makes the statement hold.

NFA to DFA

Let us assume, that the statement holds for reductions of at most n steps ($n \geq 0$) and consider a reduction $qu \Rightarrow_A^* pv$ consisting $n + 1$ steps.

Then $q'u \Rightarrow_{A'}^* p'_1 v_1 \Rightarrow_{A'} p'v$ holds, where $v_1 = av$ for some $a \in \Sigma$ and $p'_1 \in Q'$.

Then

$$p \in p' = \delta'(p'_1, a) = \bigcup_{p_1 \in p'_1} \delta(p_1, a)$$

holds, so there must be a $p_1 \in p'_1$ with $p \in \delta(p_1, a)$.

Then $p_1 v_1 = p_1 av \Rightarrow_A pv$. (1)

On the other hand, by induction there is a $q \in q'$ with

$$qu \Rightarrow_A^* p_1 v_1. \quad (2)$$

Lemma 2 follows from (1) and (2).

NFA to DFA

Lemma 2 implies $L(A') \subseteq L(A)$:

Assume, that $u \in L(A')$, i.e. $q'_0 u \Rightarrow_{A'}^* p'$ and $p' \in F'$ hold.

By the definition of F' there is a $p \in p'$ having $p \in F$. By Lemma 2 $q_0 u \Rightarrow_A^* p$ holds for some $q_0 \in q'_0 = Q_0$ implying $u \in L(A)$.

As u has been chosen arbitrarily we have $L(A') \subseteq L(A)$ proving the theorem. □

NFA to DFA

Example:

	<i>a</i>	<i>b</i>
$\rightarrow q_0$	$\{\}$	$\{q_1, q_2\}$
$\Leftrightarrow q_1$	$\{q_0\}$	$\{\}$
$\leftarrow q_2$	$\{q_1\}$	$\{q_2\}$

NFA

$\{\}$
 $\{q_0\}$
 $\leftarrow \{q_1\}$
 $\leftarrow \{q_2\}$
 $\Leftrightarrow \{q_0, q_1\}$
 $\leftarrow \{q_0, q_2\}$
 $\leftarrow \{q_1, q_2\}$
 $\leftarrow \{q_0, q_1, q_2\}$

	<i>a</i>	<i>b</i>
$\{\}$	$\{\}$	$\{\}$
$\{q_0\}$	$\{\}$	$\{q_1, q_2\}$
$\leftarrow \{q_1\}$	$\{q_0\}$	$\{\}$
$\leftarrow \{q_2\}$	$\{q_1\}$	$\{q_2\}$
$\Leftrightarrow \{q_0, q_1\}$	$\{q_0\}$	$\{q_1, q_2\}$
$\leftarrow \{q_0, q_2\}$	$\{q_1\}$	$\{q_1, q_2\}$
$\leftarrow \{q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_2\}$
$\leftarrow \{q_0, q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_1, q_2\}$

DFA

$\mathcal{L}_3$ is closed under further operations

Corollary

$\mathcal{L}_3$ is closed under set complement, intersection and difference.

Proof:

- ▶ Let $L \in \mathcal{L}_3$. According to an earlier theorem there is a deterministic finite automaton $A = (Q, \Sigma, \delta, q_0, F)$ recognizing L . Obviously, deterministic finite automaton $A' = (Q, \Sigma, \delta, q_0, Q \setminus F)$ recognizes $\bar{L}$.

Since $\bar{L}$ can be recognized by a deterministic finite automaton we have $\bar{L} \in \mathcal{L}_3$ according to an earlier theorem.

- ▶ $\mathcal{L}_3$ being closed under intersection follows from $L_1 \cap L_2 = \overline{\bar{L}_1 \cup \bar{L}_2}$ and $\mathcal{L}_3$ being closed under union and complement.
- ▶ $\mathcal{L}_3$ being closed under difference follows from $L_1 \setminus L_2 = L_1 \cap \bar{L}_2$ and $\mathcal{L}_3$ being closed under intersection and complement.

Describing the class of regular languages

All of the following concepts describe language class $\mathcal{L}_3$.

- ▶ right-linear grammars (by definition)
- ▶ left-linear grammars
- ▶ grammars of type 3 normal form
- ▶ regular expressions
- ▶ regular expressions with the following operators: complement, intersection, difference, reversal, positive closure
- ▶ deterministic finite automata
- ▶ nondeterministic finite automata

Reachable states

Definition

Let $A = (Q, \Sigma, \delta, q_0, F)$ be a deterministic finite automaton. A state $q \in Q$ is called **reachable** (from the initial state) if there is a reduction $q_0 x \Rightarrow^* q$ for some $x \in \Sigma^*$. Otherwise it is called **unreachable**.

Remark: q is reachable if and only if $\delta(q_0, x) = q$ holds for some $x \in \Sigma^*$.

Eliminating unreachable states

Define a state H recursively as follows Let

$$H_0 = \{q_0\},$$

$$H_{i+1} = H_i \cup \{r \mid \delta(q, a) = r, q \in H_i, a \in \Sigma\}, \quad i = 1, 2, \dots$$

Since $H_0 \subseteq H_1 \subseteq \dots \subseteq H_k \subseteq Q$ holds, there must a natural number k with $H_k = H_{k+1}$. Then $H_\ell = H_k$ holds for every $\ell \geq k$. Let $H := H_k$.

Then H is the set of reachable states.

Let us define a DFA $A' = (Q', \Sigma, \delta', q_0, F')$ as follows.

Let $Q' = H, F' = F \cap H, \delta' : H \times \Sigma \rightarrow H$ with the property $\delta'(q, a) = \delta(q, a)$, for every $q \in H$.

It is easy to prove, that A' has no unreachable states and recognizes the same language A .

Furthermore A' is the largest subautomaton of A having only reachable states.

Eliminating unreachable states

Example:

	a	b
$\rightarrow q_0$	q_2	q_4
$\leftarrow q_1$	q_3	q_0
$\leftarrow q_2$	q_0	q_2
q_3	q_1	q_2
q_4	q_5	q_2
$\leftarrow q_5$	q_4	q_2

$$H_0 = \{q_0\}$$

$$H_1 = \{q_0, q_2, q_4\}$$

$$H_2 = \{q_0, q_2, q_4, q_5\} =$$

$$H_3 = H$$

	a	b
$\rightarrow q_0$	q_2	q_4
$\leftarrow q_2$	q_0	q_2
q_4	q_5	q_2
$\leftarrow q_5$	q_4	q_2

Remark: This algorithm might be interpreted as a breadth first search on the transition diagram from the initial state. H consists of the elements having been in the **queue** during search.

We can apply this idea to reduce the number of states needed for the NFA to DFA construction.

NFA to DFA – reachable states only

Example:

	<i>a</i>	<i>b</i>
$\rightarrow q_0$	$\{\}$	$\{q_1, q_2\}$
$\Leftrightarrow q_1$	$\{q_0\}$	$\{\}$
$\leftarrow q_2$	$\{q_1\}$	$\{q_2\}$

Queue of states to be processed

	<i>a</i>	<i>b</i>	
$\Leftrightarrow \{q_0, q_1\}$	$\{q_0\}$	$\{q_1, q_2\}$	$\{q_0, q_1\}$
$\{q_0\}$	$\{\}$	$\{q_1, q_2\}$	$\{q_0\} \{q_1, q_2\}$
$\leftarrow \{q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_2\}$	$\{q_1, q_2\} \{\}$
$\{\}$	$\{\}$	$\{\}$	$\{\} \{q_2\}$
$\leftarrow \{q_2\}$	$\{q_1\}$	$\{q_2\}$	$\{q_2\}$
$\leftarrow \{q_1\}$	$\{q_0\}$	$\{\}$	$\{q_1\}$
			EMPTY

Algorithmic decision problems

An (algorithmic) decision problem is a problem having a typically infinite set of possible inputs (e.g., the set of natural numbers or the set of words over a given alphabet) and a binary ('yes'/'no') output.

We are looking for a general method (algorithm) that works for all possible inputs.

A problem is called (algorithmically) **decidable** if there is an algorithm terminating with the appropriate answer for every possible inputs.

Remark: This concept is to be defined more precisely in FOTOC2.

Decision problems of regular grammars

Proposition

It is decidable whether the language generated by a regular grammar is empty or not.

Proof: According to a previous theorem for any grammar G of type 3 there is a DFA A recognizing $L(G)$. Construction of A is algorithmic. Observe, that $L(A) = \emptyset$ if and only if all the accepting states of A are unreachable.

This property can be checked by a breadth first search algorithm on the transition diagram from the node corresponding to the initial state of A .

Decision problems of regular grammars

Proposition

It is decidable whether the languages generated by two regular grammars are disjoint or not.

Proof: Let the languages generated by regular grammars G_1 and G_2 be L_1 and L_2 , respectively.

According to a previous theorem $L_1 \cap L_2$ is regular as well.

Furthermore, the construction of a regular grammar for $L_1 \cap L_2$ is algorithmic, since it is a composition of the following algorithmic steps.

- ▶ constructing DFA for a regular grammar,
- ▶ given a DFA, constructing DFA for the complement language,
- ▶ constructing regular grammar for a DFA,
- ▶ constructing regular grammar for the union of the languages generated by 2 regular grammars.

According to the previous theorem, emptiness of $L_1 \cap L_2$ is decidable.

Decision problems of regular grammars

Proposition

It is decidable whether the languages generated by two regular grammars are the same or not.

Proof:

Let the languages generated by regular grammars G_1 and G_2 be L_1 and L_2 , respectively.

$L_3 = (L_1 \cap \bar{L}_2) \cup (\bar{L}_1 \cap L_2)$ is regular, so there is a regular grammar G_3 generating it. G_3 can be constructed algorithmically. (By similar reasons as in the previous proof.)

$L_1 = L_2$ if and only if $L_3 = \emptyset$. According to a previous theorem, emptiness of L_3 is decidable.

Decision problems of regular grammars

Proposition

It is decidable whether the language generated by a regular grammar is a subset of the language generated by another grammar.

Let the languages generated by regular grammars G_1 and G_2 be L_1 and L_2 , respectively.

$L_3 = (L_1 \cap \bar{L}_2) = L_1 \setminus L_2$ is regular, so there is a regular grammar G_3 generating it. G_3 can be constructed algorithmically. (By similar reasons as in a previous proof.)

$L_1 \subseteq L_2$ if and only if $L_3 = \emptyset$. According to a previous theorem, emptiness of L_3 is decidable.

Linear algorithm for the type 3 membership problem

Membership problem: Given a grammar G and $u \in \Sigma^*$. $u \stackrel{?}{\in} L(G)$.

Theorem

The membership problem of type 3 grammars is decidable.

Proof: Let $G = (N, \Sigma, P, S)$ be a grammar in type 3 normal form and let $u = t_1 \cdots t_n$ be the word to be derived.

The algorithm computes a sequence H_i ($0 \leq i \leq n$) of subsets of N recursively. H_{i+1} can be computed from H_i in constant time.

Let $H_0 := \{S\}$ $H_{i+1} := \{A \in N \mid \exists B \in H_i \wedge B \rightarrow t_{i+1}A \in P\}$.

It is easy to see, that H_i consists of exactly those nonterminals, that can be the last letter of a sentential form after i derivation steps.

So, if $F = \{A \in N \mid A \rightarrow \varepsilon \in P\}$, then we have

$$u \in L(G) \Leftrightarrow H_n \cap F \neq \emptyset.$$

Linear algorithm for type 3 membership problem

Example:

$$S \rightarrow aA \mid bS \mid \varepsilon$$

$$A \rightarrow aA \mid aS$$

For abb we have $H_0 = \{S\}$, $H_1 = \{A\}$, $H_2 = \emptyset$, $H_3 = \emptyset$,
while for aab we have $H_0 = \{S\}$, $H_1 = \{A\}$, $H_2 = \{A, S\}$, $H_3 = \{S\}$.
 $F = \{S\}$, so aab can be generated, while abb can not.

Observe, that $H_i \in \mathcal{P}(N)$ holds and H_{i+1} depends only on H_i and t_{i+1} . ($1 \leq i \leq n-1$)

I.e., computing the sets $\{H_i\}_{0 \leq i \leq n}$ can be implemented by the following DFA.

	a	b
$\Leftrightarrow \{S\}$	$\{A\}$	$\{S\}$
$\{A\}$	$\{S, A\}$	$\{\}$
$\leftarrow \{S, A\}$	$\{S, A\}$	$\{S\}$
$\{\}$	$\{\}$	$\{\}$

Linear algorithm for type 3 membership problem

Remarks:

- ▶ Observe, that this DFA is nothing else, but the DFA constructed for the NFA constructed for grammar G . The set of states is $\mathcal{P}(N)$, initial state $H_0 = \{S\}$, transition function $\delta(H, a) = \{B \in N \mid \exists A \in H : A \rightarrow aB \in P\}$ ($H \in \mathcal{P}(N)$, $a \in \Sigma$) and set of accepting states $F' = \{H \subseteq N \mid H \cap F \neq \emptyset\}$, where $F = \{A \in N \mid A \rightarrow \varepsilon \in P\}$.
- ▶ The algorithm deciding $u \stackrel{?}{\in} L(G)$ runs in **linear** time, since computing each H_i takes constant time (it depends on G , but not on $|u|$).
- ▶ If membership problem is to be asked for a long word compared to the size of G or asked for several words it might be faster to construct the above mentioned automaton in advance.
- ▶ On the other hand for a small amount of short words it might be more efficient to compute the sets H_i only if they are needed.